



Université Gaston Berger de Saint-Louis

Master Ingénierie Informatique et TIC

DBI1 - CONCEPTION DES BASES DE DONNEES

Vidéo Tutorielle : Chapitre 2 : Objets avancés 1

Objectifs

Après avoir terminé ce tutoriel vous devriez être capable de :

- Définir des tables : contraintes de tables et de colonnes
- Vous familiariser avec les principales fonctions scalaires du PL/SQL
- Maîtriser la structure d'un programme PL/SQL
- Créer des procédures et fonctions stockées
- Gérer les exceptions d'un programme PL/SQL
- Encapsuler des procédures et fonctions dans des packages



Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- La structure d'un programme PL/SQL
- Les procédures stockées
- La gestion des exceptions
- Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Environnement du Tutoriel

- Oracle Database 11g Enterprise Edition Release 11.2
64bit Production – With the Partitioning, OLAP, DataMining
- Oracle Database 11g XE
- SQL Data Modeler / SQL Developer / Oracle SQL*Plus
- Jeu d'essai de l'utilisateur **HRDBI1**



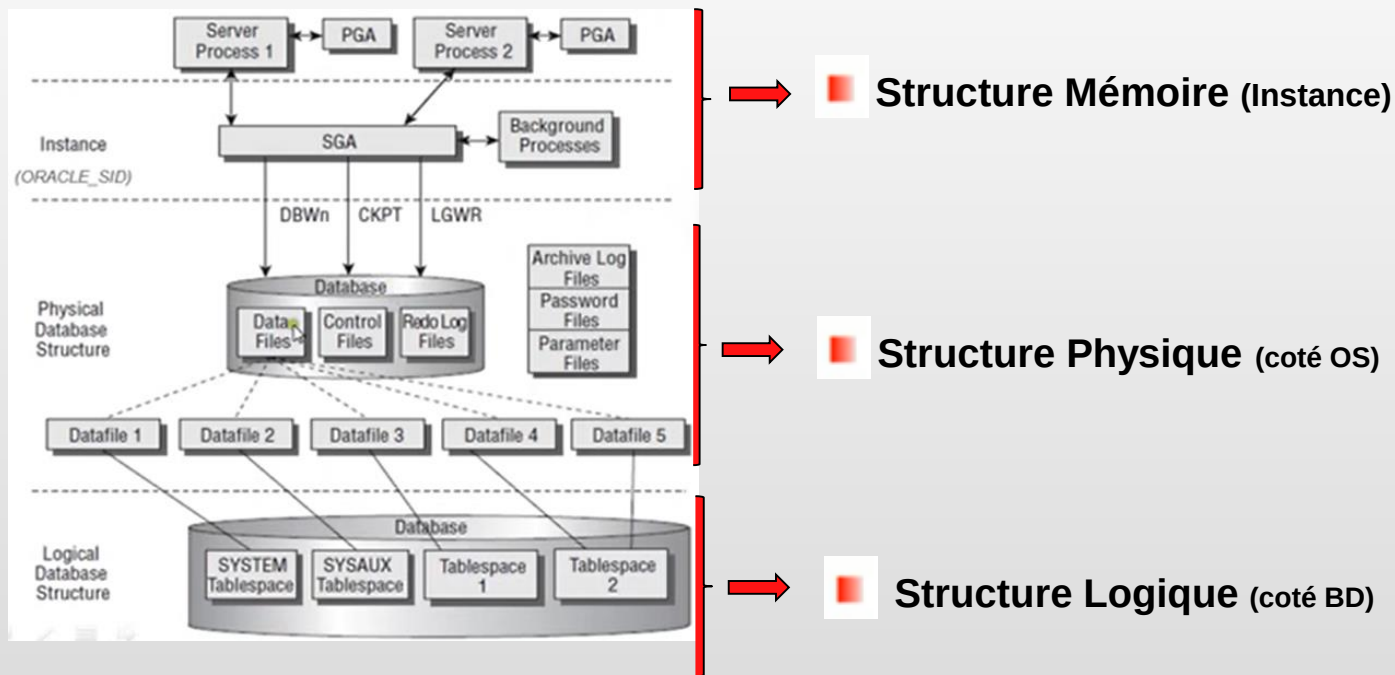
Agenda



- ✓ Les description des Objets, les contraintes
 - Les fonctions scalaires
 - La structure d'un programme PL/SQL
 - Les procédures stockées
 - La gestion des exceptions
 - Les fonctions stockées
 - Les packages (Mémoire Shared-Pool)

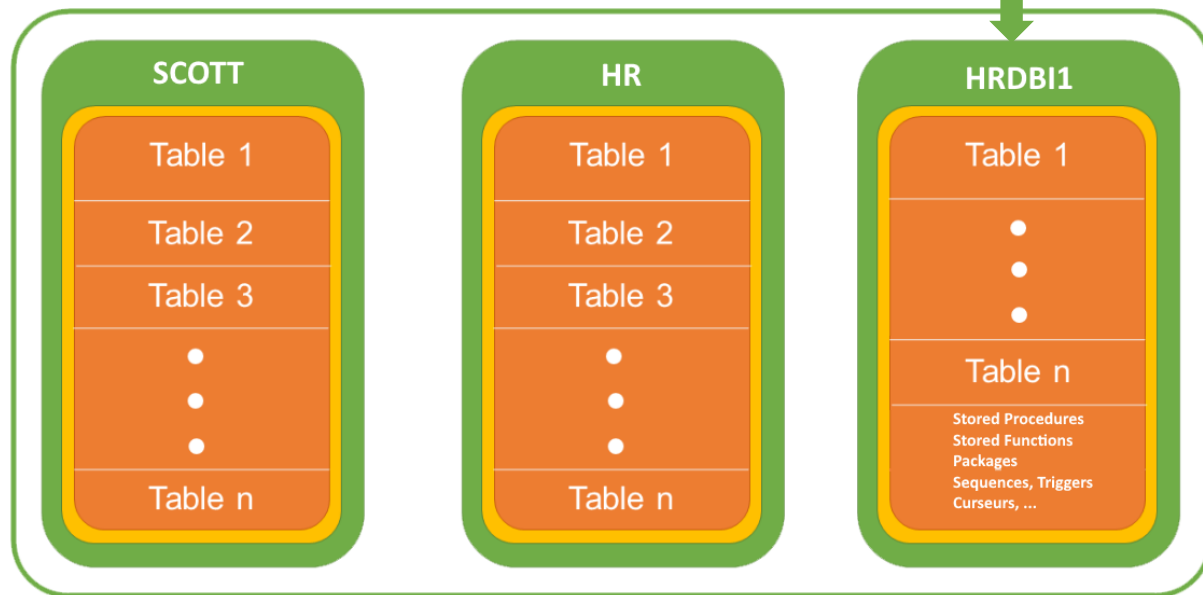
ARCHITECTURE

La structure



SCHEMAS UTILISATEURS

UFOAD-M2ITIC-DB1-CH02-HRDBI1-Table.sql
UFOAD-M2ITIC-DB1-CH02-HRDBI1-Insert.sql



Description des objets (Annexe 1)



▪ Les types de données : Exemple

```
CREATE TABLE Countries  
  (Country_id CHAR(2),  
   Country_name VARCHAR2(40),  
   Cotation NUMBER(2),  
   Region_id NUMBER);
```

Description des objets



▪ Les contraintes de colonne : Exemple

```
--- Creation de la table Country
CREATE TABLE Country
  (Countryid CHAR(2) NOT NULL,
   Country_name VARCHAR2(40) DEFAULT 'France',
   Cotation NUMBER(2)
    CONSTRAINT CHK_Cotation
      CHECK (Cotation >=0 AND Cotation <=20),
   Regionid NUMBER,
    CONSTRAINT PK_Country_CountryId
      PRIMARY KEY (CountryId));
```

Description des objets



▪ Les contraintes de table : syntaxe

```
PRIMARY KEY (colonne, ...)  
UNIQUE (colonne,...)  
FOREIGN KEY (colonne, ...) REFERENCES table [(colonne, ...)]  
[ON DELETE {CASCADE | SET NULL}] CHECK (condition)  
  
CREATE TABLE Country  
  (Countryid CHAR(2) NOT NULL,  
   Country_name VARCHAR2(40) DEFAULT 'France', (...)  
   CONSTRAINT PK_Country_Country  
     PRIMARY KEY (CountryId,Country_Name),  
   CONSTRAINT FK_Country_Region  
     FOREIGN KEY (RegionId)  
     REFERENCES Region(RegionId));
```

▪ Les contraintes de table : Exemple



```
--- Creation de la table Country
CREATE TABLE Country
    (Countryid CHAR(2) NOT NULL,
     Country_name VARCHAR2(40) DEFAULT 'France',
     Cotation NUMBER(2),
     Regionid NUMBER);

--- Implémentation des contraintes
ALTER TABLE Country
    ADD CONSTRAINT PK_Country_CountryId
    PRIMARY KEY (CountryId);
ALTER TABLE Country
    ADD CONSTRAINT FK_Country_Region
    FOREIGN KEY (RegionId)
    REFERENCES Region(RegionId);
ALTER TABLE Country
    ADD CONSTRAINT CHK_Cotation
    CHECK (Cotation >=0 AND Cotation <=20);
```

Description des objets (Annexe 1)



▪ Les colonnes virtuelles : Exemple

```
CREATE TABLE Countries  
  (Country_id CHAR(2),  
   Country_name VARCHAR2(40),  
   Cotation NUMBER(2),  
   Region_id NUMBER);
```

Description des objets : colonnes virtuelles



- **Caractéristiques :**

- Permettent de définir une expression sur une colonne qui est évaluée à chaque interrogation.

- **Syntaxe :**

```
nomcolonne [type] [GENERATED ALWAYS] AS (expression)
[VIRTUAL] [contrainte-de-colonne]
```

11^g

- **Exemple :**

```
ALTER TABLE Employee
    ADD SalTot NUMBER (8,2)
    AS (Salary * (1+Commission_Pct));
```

Description des objets

▪ Les conventions de nommage

PK_ : Clés primaires
FK_ : Clés étrangères
NN_ : Not Null
CK_ : Contraintes de Check



<http://williamrobertson.net/documents/plsqlcodingstandards.html>

Agenda



- Les description des Objets, les contraintes
- ☑ Les fonctions scalaires
- La structure d'un programme PL/SQL
- Les procédures stockées
- La gestion des exceptions
- Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Les Fonctions scalaires (Annexe 2)



▪ Caractéristiques :

- Mathématiques
- Chaîne de caractères
- Dates
- Conversion
- Comparaison

▪ Exemple :

```
SQL> SELECT TO_CHAR(SYSDATE,'HH24:MI:SS') AS Heure FROM DUAL;
```

```
HEURE
```

```
-----
```

```
18:21:02
```

```
SQL>
```

Les instructions DML (Annexe 3)



▪ Caractéristiques :

- Insertion de données
- Modification de données
- Suppression de données

▪ Exemple :

```
SQL> TRUNCATE TABLE Essai;
```

```
Table truncated.
```

Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- ✓ La structure d'un programme PL/SQL
- Les procédures stockées
- La gestion des exceptions
- Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Structure d'un programme PL/SQL



▪ Syntaxe :

DECLARE

(déclaration des variables, des constantes,
des exceptions et des curseurs)

BEGIN

(instruction SQL, PL/SQL, structures de contrôle)

EXCEPTION

(traitement des erreurs)

END;

▪ Exemple :



```
SQL> var vl_nbregion number
SQL> variable
variable    vl_nbregion
type de données    NUMBER
SQL> BEGIN
  2      SELECT COUNT(*) INTO :vl_nbregion
  3      FROM Region;
  4  END;
  5  /
```

Procédure PL/SQL terminée avec succès.

```
SQL> print vl_nbregion
VL_NBREGION
-----
          4

SQL>
```

Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- La structure d'un programme PL/SQL
- ☑ Les procédures stockées
- La gestion des exceptions
- Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Les procédures stockées (Stored Procedure)



▪ Caractéristiques :

- Blocs de codes PL/SQL nommés, compilés
- Stockées dans la base de données : traitements complexes
- Appelées via leur nom à partir du code ou d'une autre procédure ou fonction.

▪ Syntaxe :

```
CREATE [OR REPLACE] PROCEDURE procedure_name  
[(parameter_name [IN | OUT | IN OUT] type [, ...])]  
{IS | AS}  
BEGIN  
    procedure_body  
END procedure_name;
```

Les procédures stockées



- **Appel :**

- Avec l'instruction CALL

- **Syntaxe :**

```
CALL procedure_name (Parameter,...);
```

```
CALL procedure_name(Named_Notation=>Parameter);
```

11^g

▪ Exemple 1:



```
CREATE OR REPLACE PROCEDURE sp_SalaryTotal (p_DepartmentId IN
NUMBER) AS

vl_SalaryTotal Employee.Salary%TYPE;

-- Total des salaires pour un département donné

BEGIN
SELECT SUM(Salary) INTO vl_SalaryTotal
      FROM Employee
      WHERE DepartmentId = p_DepartmentId;

DBMS_OUTPUT.PUT_LINE('Total des salaires pour le département :'||
TO_CHAR (p_DepartmentId)||' = '||TO_CHAR (vl_SalaryTotal));

END;
```

■ Exemple 2:



```
CREATE OR REPLACE PROCEDURE sp_SalaryIncrease (p_EmployeeId IN NUMBER, p_CoefIncrease IN
NUMBER) AS

vl_Hire_Date DATE;
vl_Salary Employee.Salary%TYPE;
vl_Last_Name Employee.Last_Name%TYPE;
-- Augmentation d'un salaire en fonction d'un coef seulement si embauche avant 01/01/2000
BEGIN
SELECT Hire_Date,Salary,Last_Name INTO vl_Hire_Date,vl_Salary,vl_Last_Name
      FROM Employee
      WHERE EmployeeId = p_EmployeeId;
IF vl_Hire_Date < TO_DATE ('01/01/2000','DD/MM/YYYY') THEN
    vl_Salary := vl_Salary * p_CoefIncrease;
    UPDATE Employee SET Salary = vl_Salary WHERE EmployeeId=p_EmployeeId;
    DBMS_OUTPUT.PUT_LINE('Salarie :'|| vl_Last_Name||' salaire augmente a '||TO_CHAR
    (vl_Salary));
    COMMIT;
ELSE
    DBMS_OUTPUT.PUT_LINE('Le Salarie :'|| vl_Last_Name||' a ete embauche le
    '||TO_CHAR (vl_Hire_Date,'DD/MM/YYYY')||'. Il ne sera pas augmente.');
```

```
END IF;

END sp_SalaryIncrease;
```

Les procédures stockées



▪ Informations :

```
SELECT Object_Name, Procedure_Name, Object_Type  
FROM User_Procedures;
```

▪ Visualisation des erreurs

```
SHOW ERRORS
```

▪ Droits

```
GRANT EXECUTE ON procedure_name TO <user>|<role>;  
REVOKE EXECUTE ON procedure_name FROM <user>|<role>;
```

▪ Suppression :

```
DROP PROCEDURE procedure_name;
```

Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- La structure d'un programme PL/SQL
- Les procédures stockées
- ✓ La gestion des exceptions
- Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Les Exceptions prédéfinies (Annexe 4)



▪ Caractéristiques :

- possèdent un numéro d'identification unique (ex : ORA-01403 NO_DATA_FOUND)
- peuvent être interceptées pour les plus courantes via leur nom
- appartiennent au package standard donc utilisable telles quelles dans les programmes PL/SQL

▪ Syntaxe :

DECLARE

(déclaration des variables, des constantes, et des curseurs)

BEGIN

(instruction SQL, PL/SQL, structures de contrôle)

EXCEPTION

WHEN nom_exception THEN ...;

END;



▪ Exemple :



```
CREATE OR REPLACE PROCEDURE sp_SalaryIncrease (p_EmployeeId IN NUMBER,  
p_CoefIncrease IN NUMBER) AS  
  
    vl_Hire_Date DATE;  
    vl_Salary Employee.Salary%TYPE;  
    vl_Last_Name Employee.Last_Name%TYPE;  
    -- Augmentation d'un salaire en fonction d'un coef seulement si embauche  
    avant 01/01/2000  
  
BEGIN  
    SELECT Hire_Date,Salary,Last_Name INTO  
    vl_Hire_Date, vl_Salary, vl_Last_Name  
        FROM Employee  
        WHERE EmployeeId = p_EmployeeId;  
  
    (...)  
EXCEPTION  
WHEN NO_DATA_FOUND THEN  
    DBMS_OUTPUT.PUT_LINE('ATTENTION : Ce salaire n''existe pas');  
END sp_SalaryIncrease;
```

Les Exceptions liées au programme utilisateur



▪ Caractéristiques :

- doivent être déclarées dans la section DECLARE du bloc PL/SQL
- sont définies avec un nom suivi du mot clé EXCEPTION (ex :
NOT_INCREASE EXCEPTION;)
- sont appelées explicitement par l'instruction RAISE

▪ Syntaxe :



DECLARE

(déclaration des variables, des constantes,
des exceptions et des curseurs)

nom_exception EXCEPTION;

BEGIN

(instruction SQL, PL/SQL, structures de contrôle)

IF condition THEN nom_exception; END IF;

...

EXCEPTION

WHEN nom_exception THEN ...;

END;



▪ Exemple :

```
CREATE OR REPLACE PROCEDURE sp_SalaryIncrease (p_EmployeeId IN
NUMBER, p_CoefIncrease IN NUMBER) AS
(...)
NOT_INCREASE EXCEPTION;
BEGIN
(...)
IF vl_Hire_Date < TO_DATE ('01/01/2000','DD/MM/YYYY') THEN
    (...)
ELSE
    RAISE NOT_INCREASE;
END IF;
(...)
EXCEPTION
WHEN NOT_INCREASE THEN
    DBMS_OUTPUT.PUT_LINE('Ce salaire n''a pas assez d''anciennete');
    (...)
END sp_SalaryIncrease;
```

Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- La structure d'un programme PL/SQL
- Les procédures stockées
- La gestion des exceptions
- ☒ Les fonctions stockées
- Les packages (Mémoire Shared-Pool)

Les fonctions stockées (Stored Functions)



▪ Caractéristiques :

- Blocs de codes PL/SQL nommés, compilés qui **renvoie une valeur**
- Stockées dans la base de données : traitements complexes
- Appelées via leur nom à partir du code ou d'une autre procédure ou d'une autre fonction.

▪ Syntaxe :

```
CREATE [OR REPLACE] FUNCTION function_name  
[(parameter_name [IN] type, ...)]  
RETURN type {IS/AS};  
BEGIN  
    function_body  
    RETURN type {IS/AS};  
END function_name;
```

Les fonctions stockées



▪ Appel :

- Comme pour une fonction intégrée dans la base de données (via la pseudo table DUAL)

▪ Syntaxe :

```
SELECT function_name (Parameter,...)  
FROM DUAL;
```

```
SELECT function_name (Named_Notation=>Parameter)  
FROM DUAL;
```

11^g

▪ Example 1 :



```
CREATE OR REPLACE FUNCTION fn_circle_area (p_radius IN NUMBER) RETURN
NUMBER AS
    vl_pi    NUMBER := 3.1415926;
    vl_area  NUMBER;
BEGIN
    -- circle area is pi multiplied by the radius squared
    vl_area := vl_pi * POWER(p_radius, 2);
    RETURN vl_area;
END fn_circle_area;
```



▪ Exemple 2 :

```
CREATE OR REPLACE FUNCTION fn_Salary_Average (  
    p_Departmentid Department.Departmentid%TYPE) RETURN NUMBER AS  
    vl_Salary_average NUMBER;  
  
BEGIN  
    SELECT ROUND(AVG(Salary),0) INTO vl_Salary_Average  
    FROM Employee  
    WHERE Departmentid = p_Departmentid;  
  
    RETURN vl_Salary_Average;  
  
END fn_Salary_Average;  
/
```

Les fonctions stockées



▪ Informations :

```
SELECT Object_Name, Procedure_Name, Object_Type  
FROM User_Procedures;
```

▪ Visualisation des erreurs

```
SHOW ERRORS
```

▪ Droits

```
GRANT EXECUTE ON function_name TO username;  
REVOKE EXECUTE ON function_name FROM username;
```

▪ Suppression :

```
DROP FUNCTION function_name;
```

Agenda



- Les description des Objets, les contraintes
- Les fonctions scalaires
- La structure d'un programme PL/SQL
- Les procédures stockées
- La gestion des exceptions
- Les fonctions stockées
- ☒ Les packages (Mémoire Shared-Pool)

Les packages

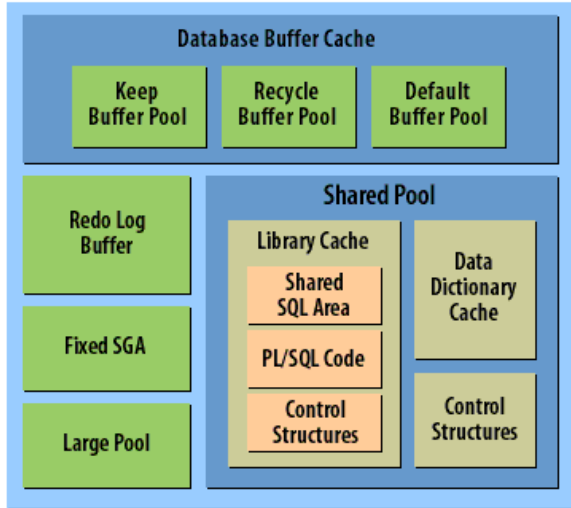


▪ Caractéristiques :

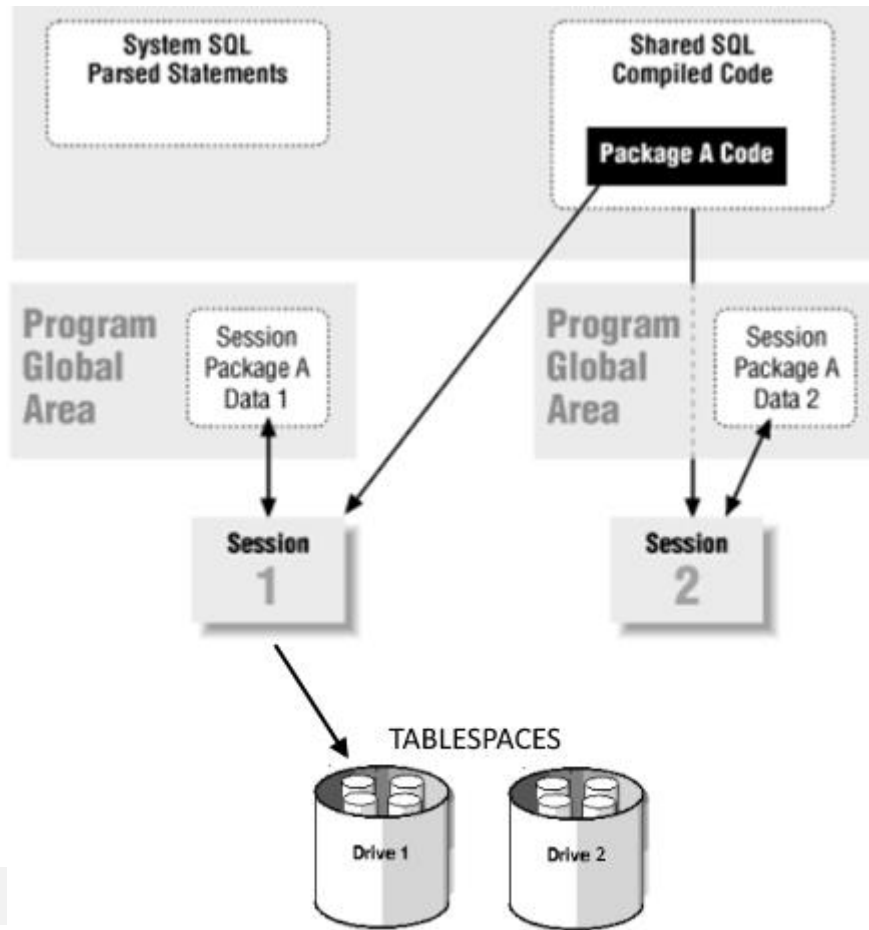
- Permettent d'encapsuler d'autres objets du PL/SQL (types de données, procédures, fonctions, curseurs)
- Permet de structurer logiquement ces objets
- Composés d'une entête et d'un corps (body)
- Facilitent la gestion des droits d'exécution
- Permettent de spécifier quels éléments seront visibles et accessibles de ceux masqués et inaccessibles
- Restent en mémoire dès le 1^{er} appel

Les packages

▪ Fonctionnement :



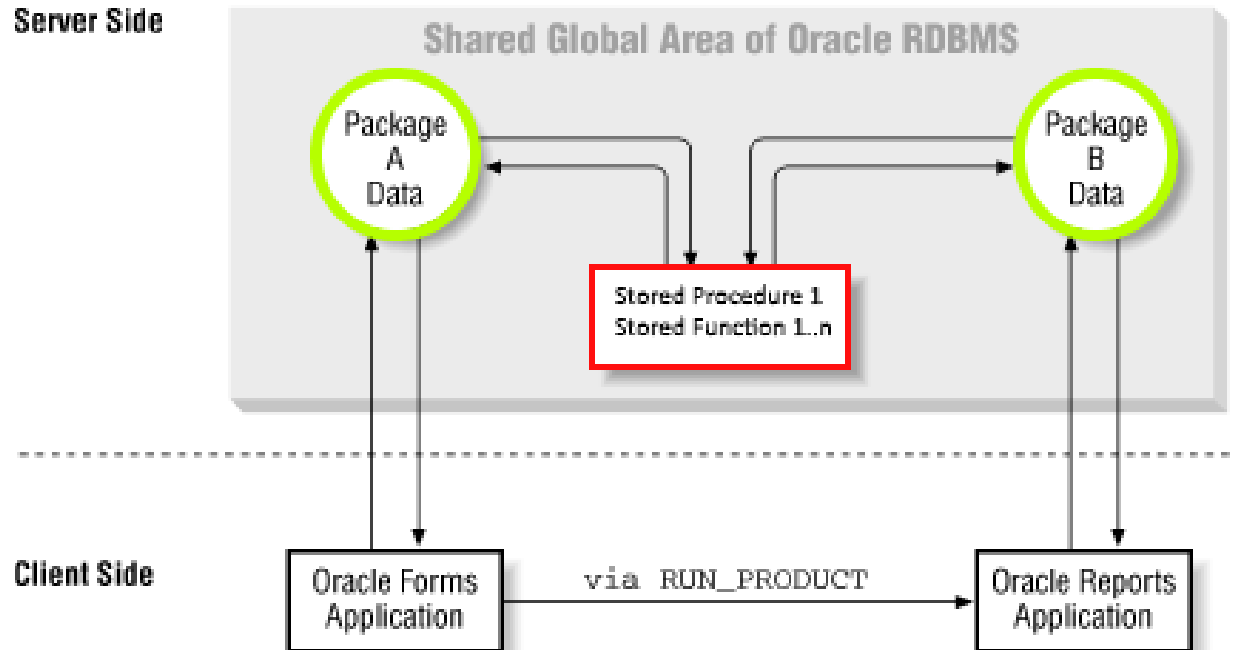
SHOW PARAMETER SHARED_POOL



Les packages



▪ Fonctionnement :



Les packages

▪ Header Syntaxe :

```
CREATE [OR REPLACE] PACKAGE package_name  
{IS | AS}  
    package_specification
```

- Définition de type
- Déclarations de variables publiques
- Prototypes des curseurs publiques
- Prototypes des procédures et fonctions publiques

```
END package_name;
```



Les packages

▪ Body Syntaxe :

```
CREATE [OR REPLACE] PACKAGE BODY package_name  
{IS | AS}
```

```
package_body
```

- Définitions de type locaux au package
- Déclarations de variables locales au package
- Implémentation des curseurs publics
- Corps des procédures et fonctions locales au package
- Corps des procédures et fonctions publiques

```
END package_name;
```



Les packages



▪ Exemple :

```
CREATE OR REPLACE PACKAGE pkg_hr AS

    FUNCTION fn_Salary_Average (p_DepartmentId Department.DepartmentId%TYPE)
        RETURN NUMBER;
    PROCEDURE sp_SalaryIncrease (p_EmployeeId IN NUMBER,
        p_CoefIncrease IN NUMBER);
    PROCEDURE sp_SalaryTotal (p_DepartmentId IN NUMBER);

END pkg_hr;
/
```

```
CREATE OR REPLACE PACKAGE BODY pkg_hr AS
```

```
FUNCTION fn_Salary_Average (  
    p_Departmentid Department.Departmentid%TYPE) RETURN NUMBER AS  
    vl_Salary_average NUMBER;  
BEGIN  
    SELECT ROUND(AVG(Salary),0)  
    INTO vl_Salary_Average  
    FROM Employee  
    WHERE Departmentid = p_Departmentid;  
    RETURN vl_Salary_Average;  
END fn_Salary_Average;
```

```
PROCEDURE sp_SalaryIncrease (p_EmployeeId IN NUMBER, p_CoefIncrease IN  
NUMBER) AS
```

```
    vl_Hire_Date DATE;  
    vl_Salary Employee.Salary%TYPE;  
    vl_Last_Name Employee.Last_Name%TYPE;
```



```
-- Augmentation d'un salaire en fonction d'un coef seulement si embauche  
avant 01/01/2000
```



```
BEGIN  
SELECT Hire_Date,Salary,Last_Name INTO vl_Hire_Date,vl_Salary,vl_Last_Name  
      FROM Employee  
      WHERE EmployeeId = p_EmployeeId;  
IF vl_Hire_Date < TO_DATE ('01/01/2000','DD/MM/YYYY') THEN  
    vl_Salary := vl_Salary * p_CoefIncrease;  
    UPDATE Employee SET Salary = vl_Salary WHERE  
      EmployeeId=p_EmployeeId;  
    DBMS_OUTPUT.PUT_LINE('Salarie :'|| vl_Last_Name||' salaire  
      augmente a '||TO_CHAR (vl_Salary));  
COMMIT;  
ELSE  
    DBMS_OUTPUT.PUT_LINE('Le Salarie :'|| vl_Last_Name||' a ete  
      embauche le '||TO_CHAR (vl_Hire_Date,'DD/MM/YYYY')||'. Il ne sera  
      pas augmente.');
```

```
END IF;
```



```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Ce salaire n''existe pas');
```

```
END sp_SalaryIncrease;
```

```
PROCEDURE sp_SalaryTotal (p_DepartmentId IN NUMBER) AS  
vl_SalaryTotal Employee.Salary%TYPE;  
BEGIN
```

```
SELECT SUM(Salary) INTO vl_SalaryTotal
```

```
FROM Employee
```

```
WHERE DepartmentId = p_DepartmentId;
```

```
DBMS_OUTPUT.PUT_LINE('Total des salaires pour le d,partement :'|| TO_CHAR  
(p_DepartmentId)||' = '||TO_CHAR (vl_SalaryTotal));
```

```
END;
```

```
END pkg_hr;
```

```
/
```

Les packages



- Appel d'une fonction appartenant à un package :

```
SELECT package_name.function_name (parameters,...) FROM DUAL;
```

- Appel d'une procédure appartenant à un package :

```
CALL package_name.procedure_name (parameters,...) ;
```

Les packages

▪ Appel : Exemple



```
SQL> CALL pkg_hr.sp_SalaryTotal (60);  
Total des salaires pour le departement :60 = 28800
```

Appel terminé.

```
SQL>
```

```
SQL> SELECT pkg_hr.fn_Salary_Average(60) As Moyenne FROM  
DUAL;
```

```
      MOYENNE  
-----  
          5760
```

```
SQL>
```

Les packages



▪ Informations :

```
SELECT Object_Name, Procedure_Name, Object_Type  
FROM User_Procedures  
WHERE Object_Name='PRODUCT_PACKAGE';
```

▪ Visualisation des erreurs

```
SHOW ERRORS
```

▪ Droits

```
GRANT EXECUTE ON package_name TO username;  
REVOKE EXECUTE ON package_name FROM username;
```

▪ Suppression :

```
DROP PACKAGE package_name;
```

Pour aller plus loin ...



SQL pour Oracle – Christian Soutou - Edition Eyrolles
Application avec Java, PHP et XML
547 pages – (Pdf) – 14 167 Ko



Oracle Database 11g SQL – Jason Price – Oracle Press
Master SQL and PL/SQL in the Oracle Database
656 pages – (Pdf) – 3 600 Ko



Agenda



- ✓ Les description des Objets, les contraintes
- ✓ Les fonctions scalaires
- ✓ La structure d'un programme PL/SQL
- ✓ Les procédures stockées
- ✓ La gestion des exceptions
- ✓ Les fonctions stockées
- ✓ Les packages (Mémoire Shared-Pool)

Questions ?



Fin !

GET READY FOR THE FUTURE

